

TEMPLATE PADRÃO ÚNICO DO DESAFIO PROFISSIONAL

ORIENTAÇÕES IMPORTANTES ANTES DE COMEÇAR:

Este é o template padrão único para a realização do seu Desafio Profissional. Para todas as disciplinas, o template será o mesmo. O que muda é a proposta do seu desafio.

Portanto, para que você conheça o desafio proposto para a sua disciplina, é preciso:

- 1) Acessar o seu AVA;
- 2) Clicar na disciplina que será avaliada;
- 3) Entrar em “Notas e Avaliações”;
- 4) Clicar em “Responder Avaliação III”.

Além disto, é fundamental que você faça a leitura atenta da questão na íntegra antes de iniciar o preenchimento deste template.

Agora, vamos às etapas de realização do seu desafio profissional.

ETAPA 1: Apresentação do Desafio Profissional

Seu papel ativo nesta etapa é apenas ler tudo com atenção e entender qual solução (ou soluções) você apresentará ao final da atividade. Então, leia todas as orientações da Etapa 1 do seu Desafio Profissional.

ETAPA 2: Materiais de referência (ambientação) do seu Desafio Profissional

Nesta etapa, você deve analisar os materiais de referência e inteirar-se do conteúdo que o(a) professor(a) indicou para que você tenha mais segurança e conhecimento na hora de analisar o caso. Depois que você tiver feito a leitura e já estiver munido de mais informações, você deve eleger três aspectos do desafio proposto que sejam os mais relevantes, do seu ponto de vista, para a solução do desafio. Por exemplo: que estratégia inovadora foi usada? Que decisão polêmica ou uma atitude inesperada você localizou? Qual foi o erro do profissional que aplicou a fórmula? O que o profissional esqueceu de observar? **Seu papel ativo nesta etapa** é apontar esses três aspectos e justificar suas escolhas.

Estudante, escreva aqui os três aspectos e justifique suas escolhas. Anote assim neste template: o que chamou atenção + por quê.

O problema exposto no desafio é que o programador anterior usou "estratégias estáticas" para problemas que exigem "soluções dinâmicas". O que chamou atenção:

1. O uso de alocação estática em um servidor com ampla memória RAM.

Por quê: O Relatório de Memória afirma que o servidor tem 64 GB de RAM, mas o jogo diz que está "cheio" usando apenas 2 MB. Isso mostra um erro grave de arquitetura. Em vez de aproveitar a memória disponível conforme a demanda (alocação dinâmica), o código limita o crescimento do jogo a um tamanho fixo (alocação estática) definido no momento da compilação, o que é inaceitável para um jogo online que pretende escalar.

2. A ausência do conceito FIFO (First-In, First-Out) na fila de login.

Por quê: A reclamação do jogador no fórum é uma indicação clara de uma fila mal implementada. Em sistemas de espera, a ordem de chegada deve ser respeitada. O fato de novos jogadores "furarem a fila" mostra que o sistema não está usando uma estrutura de Fila (Queue) correta, gerando uma experiência injusta e frustrante para o usuário.

3. O risco de corrupção de dados e perda de informação por falha no código.

Por quê: O snippet de código mostra uma função que sobrescreve dados sem verificar se a posição está livre. Ou seja, o sistema simplesmente "atropela" os dados antigos para colocar novos, sem checar se aquele espaço já estava ocupado. Isso é muito grave, pois além de apagar informações importantes dos jogadores, faz com que o servidor pare de funcionar de repente (crash). Isso mostra que o jogo precisa de uma estrutura mais inteligente, que saiba organizar os espaços de memória conforme eles são ocupados.

ETAPA 3: Levantamento de conceitos teóricos

Aqui, você deve aproximar a teoria da prática. **Seu papel ativo nesta etapa** é pesquisar conceitos, autores, teorias etc., que possibilitem a compreensão da solução do desafio. Você pode usar o seu livro da disciplina ou ainda o material apresentado na etapa 2. Para isto, faça uma lista comentada de conceitos-chave, cada um explicado em duas ou três

linhas. Por exemplo: Nome do conceito → definição curta → como ajuda a entender o caso. Lembre-se de que é como montar uma “maleta de ferramentas teóricas” para usar na próxima etapa.

Para solucionar as falhas de desempenho e lógica do servidor do jogo "Kingdoms Online", foram levantados quatro conceitos fundamentais da disciplina de Estrutura de Dados:

***Alocação Dinâmica de Memória:** Diferente da alocação estática (onde o tamanho do vetor é fixo e definido antes do programa rodar), a alocação dinâmica permite reservar e liberar espaço na memória RAM durante a execução do sistema, utilizando ponteiros e funções específicas (como *malloc* e *free* em C, ou *new* e *delete* em C++). Isso permite que o programa use apenas a memória necessária, crescendo conforme a demanda.

***Listas Encadeadas (Estruturas Dinâmicas):** São estruturas de dados lineares onde os elementos (chamados de *nós*) não estão guardados em posições seguidas na memória. Em vez disso, cada nó possui o seu dado e um "ponteiro" que indica onde está o próximo elemento. Isso permite inserir ou remover itens a qualquer momento sem precisar empurrar os outros elementos ou limitar um tamanho máximo.

***Conceito FIFO (First-In, First-Out) em filas:** É uma estrutura de dados restrita onde o primeiro elemento a entrar deve ser o primeiro a sair. Novas informações sempre entram no final da fila, e as remoções acontecem sempre no início, garantindo uma ordem de atendimento justa e sequencial.

***Conceito LIFO (Last-In, First-Out) em pilhas:** É uma estrutura de dados invertida, onde o último elemento a entrar é o primeiro a ser retirado. Podemos pensar nela como uma pilha de pratos: sempre colocamos novos itens no topo e só conseguimos retirar o elemento que está no topo. É a estrutura ideal para rastrear históricos de ações que precisam ser revertidas.

ETAPA 4: Aplicação dos conceitos teóricos ao Desafio Profissional

Neste momento, você deve começar a construção da sua análise. É aqui que você vai usar sua “maleta de ferramentas” para solucionar o desafio. **Seu papel ativo** nesta etapa é aplicar cada conceito que julgue importante e conectá-lo com algo que acontece na situação analisada. Você fará isso por meio de uma lista de tópicos, respondendo:

- Como o conceito X explica o que aconteceu na situação Y?
- O que a teoria X nos ajuda a entender sobre o problema central?
- Que soluções possíveis a teoria aponta (e por que elas fazem sentido)?

Estrutura de Fila e o Princípio FIFO (First-In, First-Out):

O conceito de FIFO explica diretamente a injustiça na fila de login reclamada pelos jogadores no fórum. Como o programador anterior não utilizou uma estrutura de Fila correta, o servidor escolhia quem entrava de forma aleatória ou aceitava quem chegou por último. A falta da regra FIFO fez com que o sistema ignorasse a ordem de chegada, permitindo a famosa "furada de fila". O princípio FIFO garante que o primeiro jogador a solicitar o acesso seja o primeiro a ser atendido, mantendo a ordem.

Alocação Dinâmica de Memória vs. Vetores Estáticos:

O conceito de alocação dinâmica explica por que o servidor travava no 101º jogador mesmo tendo 64 GB de RAM livres. O desenvolvedor anterior limitou o lobby criando um vetor fixo (Jogador lobby[100]). Vetores estáticos têm tamanho imutável definido antes do jogo rodar, portanto, quando o limite é atingido, o programa estoura a capacidade daquele espaço reservado e trava, ignorando o resto da memória RAM disponível. A solução é substituir o vetor por uma Lista Encadeada Dinâmica, utilizando funções de alocação de memória (como malloc ou new). Isso faz sentido porque o lobby deixa de ter um tamanho fixo: cada vez que um jogador entra, o sistema reserva um pequeno bloco de RAM na hora para ele, permitindo que o lobby cresça de forma infinita, limitado apenas pela memória real do servidor.

Estrutura de Pilha e o Princípio LIFO (Last-In, First-Out):

O conceito de Pilha explica o fracasso do botão "Desfazer". O programador anterior tentou criar um histórico de ações usando uma lógica confusa e lenta. Para reverter um estado em um jogo ou software, precisamos acessar imediatamente a última ação realizada. Assim

sendo, a estrutura de Pilha (Stack) é a única solução viável nesse caso. Ela faz sentido porque funciona sob a regra do LIFO (o último que entra é o primeiro que sai). Toda jogada que o usuário faz é "empilhada" (push) no topo do histórico. Quando ele clica em "Desfazer", o sistema simplesmente remove (pop) a ação que está no topo, que é exatamente a última jogada que ele fez, restaurando o jogo instantaneamente.

A ETAPA 5 É A MAIS IMPORTANTE DE TODO O PROCESSO, POIS É A ETAPA QUE SERÁ AVALIADA! ENTÃO, PRESTE MUITA ATENÇÃO!

ETAPA 5 – AVALIATIVA: Redação do produto - Memorial Analítico.

Chegou a hora de transformar todo o seu percurso investigativo em um texto claro, bem estruturado e objetivo. **Seu papel ativo nesta etapa** é desenvolver um **Memorial Analítico**. Este será o produto final do Desafio Profissional, que será avaliado com nota de zero a dez e terá peso três na média final desta disciplina.

Vamos reforçar o que é um memorial analítico? É basicamente você mostrando o caminho que percorreu: o que leu, como interpretou, que teorias usou, que conclusões tirou e o que aprendeu com tudo isso.

Para ajudar você, segue o passo a passo do que não pode faltar no Memorial Analítico (ordem recomendada, pois cada item fará parte da composição da sua nota):

- Resumo do que você descobriu (1 parágrafo) – **vale 1 ponto**
- Contextualização do desafio (1 parágrafo): Quem? Onde? Qual a situação? – **vale 0,5 ponto**
- Análise (1 parágrafo): use de 2 a 3 conceitos da disciplina, mostrando como eles explicam a situação. Dê exemplos diretos e contextualizados – **vale 2 pontos**
- Propostas de solução (até 2 parágrafos): o que você recomenda? Por quê? Qual teoria apoia sua ideia? – **vale 3 pontos**
- Conclusão reflexiva (até 2 parágrafos): O que você aprendeu com essa experiência? – **vale 2 pontos**

- Referências (somente o que você realmente usou, incluindo o livro) – **vale 0,5 ponto**
- Autoavaliação (1 parágrafo): o que você percebeu sobre seu próprio processo de estudo? – **vale 1 ponto**

Checklist rápido antes de entregar:

- Meu texto não passou de 6000 caracteres.
- Meus conceitos fazem sentido, e não estão só “porque sim”.
- Conectei teoria + situação.
- Apresentei soluções plausíveis.
- Incluí referências.
- Mostrei que aprendi algo.
- Tenho orgulho do que escrevi.

Lembre-se de que este trecho deve ser copiado e colado no campo de resposta da questão, dentro de Notas e Avaliações.

Lembre-se também de salvar este documento em PDF e colocá-lo como anexo à sua resposta.

Memorial Analítico

Resumo: A investigação do código do jogo “Kingdoms Online” revelou que a escolha errada das estruturas de armazenamento pode inviabilizar completamente o desempenho de um software. Pude perceber que o uso de vetores estáticos e desordem de lógica no código cria limitações que travam o sistema, mesmo com hardware de sobra. A solução para esses problemas está na transição para estruturas dinâmicas, como Listas Encadeadas, Pilhas e Filas, que utilizam a alocação dinâmica de memória para garantir estabilidade, justiça no atendimento aos usuários e escalabilidade infinita para a aplicação.

Contextualização: Fui contratada como engenheira de software sênior pela PixelStorm Games para solucionar falhas críticas no servidor do jogo de estratégia “Kingdoms Online”. O backend do jogo, desenvolvido em C/C++, apresentava três problemas graves vindos de uma programação amadora: um lobby de espera limitado a 100 jogadores que derrubava o

servidor, uma fila de login injusta onde usuários novos furavam a fila de quem já esperava há horas, e um botão "Desfazer" extremamente lento e confuso para o histórico de jogadas.

Análise: Para compreender as falhas do sistema, utilizei conceitos centrais da disciplina. O travamento do lobby no 101º jogador explica-se pela limitação da alocação estática, que aprisionou o jogo em um vetor fixo e desperdiçou os 64 GB de RAM do servidor. Já a injustiça no login ocorreu pela total ausência do princípio FIFO (First-In, First-Out), que deveria organizar os acessos por ordem de chegada. Por fim, a lentidão do botão "Desfazer" aconteceu porque o histórico de ações não seguia a lógica de uma Pilha, onde o princípio LIFO (Last-In, First-Out) é essencial para acessar imediatamente a última jogada realizada. Na prática, percebi que sem alinhar a teoria das estruturas ao comportamento esperado do jogo, o software entra em colapso.

Propostas de Solução: A estratégia principal que adotei para reestruturar o servidor foi a substituição completa das soluções estáticas por estruturas dinâmicas de dados. Para o problema do lobby, eliminei o vetor fixo e implementei uma Lista Encadeada Dinâmica utilizando ponteiros. Isso significa que o lobby agora deverá crescer de forma flexível: cada vez que um jogador se conecta, o sistema aloca memória sob demanda, eliminando o

limite rígido de 100 usuários.

```
6
7 // Estrutura que representa o jogador
8 struct Jogador {
9     int id;
10    std::string nome;
11 };
12
13 // Estrutura do Nó da Lista Encadeada
14 struct NoLobby {
15     Jogador dado;
16     NoLobby* proximo; // Ponteiro para o próximo jogador da lista
17 };
18
19 // Função para adicionar jogador no lobby (Alocação Dinâmica)
20 void entrarNoLobby(NoLobby*& topo, int id, std::string nome) {
21     // Aloca memória dinamicamente para um novo nó
22     NoLobby* novoNo = new NoLobby();
23     novoNo->dado.id = id;
24     novoNo->dado.nome = nome;
25
26     // Insere no início da lista (Lobby)
27     novoNo->proximo = topo;
28     topo = novoNo;
29
30     std::cout << nome << " entrou no lobby com sucesso!\n";
31 }
```

Além disso, propus a organização da fila de login através de uma Fila Dinâmica estruturada, onde novos jogadores entram estritamente no final (enqueue) e o servidor libera o acesso apenas para quem está no início da fila (dequeue), restabelecendo a justiça no servidor. Para o botão "Desfazer", a solução ideal foi a criação de uma Pilha Dinâmica, que armazena as ações de trás para frente. Assim, quando o jogador solicita o cancelamento de uma jogada, o sistema remove o elemento do topo da pilha, restaurando o estado imediatamente anterior do jogo de forma instantânea e sem lentidão.

Para demonstrar a eficiência dessas soluções, disponibilizei a modelagem lógica das estruturas no meu repositório: <https://github.com/michellevillaca/desafio-profissional>.

Conclusão Reflexiva: Desenvolver este projeto de remodelação expandiu minha visão sobre o impacto do código no mundo real. Ver como uma decisão ruim de estrutura de dados pode destruir a experiência de milhares de jogadores e inutilizar um servidor potente me fez entender que programar não é apenas fazer o software "funcionar", mas sim otimizar recursos com inteligência e eficiência.

A maior lição veio de enfrentar a complexidade dos ponteiros e da gerência de memória em C/C++. Entender as regras de CORS na matéria passada foi um desafio de conectividade, mas lidar com alocação dinâmica me ensinou a ter um olhar muito mais analítico sobre como o computador organiza as informações internamente. Sinto que desenvolvi a resiliência para ler códigos confusos, diagnosticar o erro estrutural e aplicar a teoria certa para salvar o projeto.

Referências:

OLIVEIRA, P. M.; PEREIRA, R. L. Estrutura de Dados. Indaial: UNIASSELVI, 2023.

SINGER, H. Alocação dinâmica de memória, Listas Encadeadas, Pilhas e Filas. Vídeos de apoio da disciplina. 2026.

Autoavaliação: Ao longo deste desafio de Estrutura de Dados, percebi que meu processo de estudos ficou mais maduro. Vejo que não basta focar apenas na sintaxe das linguagens, devo também avaliar o impacto de estrutura de cada escolha, como a diferença gritante entre memória estática e dinâmica. Notei que interpretar as reclamações dos usuários no fórum e traduzi-las em conceitos como FIFO e LIFO foi um exercício excelente de raciocínio lógico.